

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case Study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Questions Attempted: 4 (of 6) — 10 Marks each — Total 40

Name / Reg No

Girija B - 192311044

Code of Conduct

I, GIRIJA B/192311044, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Girija B

Question 1: Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions. Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

1. Identified Classes

- User (base class)
- Customer
- HotelManager
- Administrator
- Hotel
- Room
- Booking
- Payment
- Review

2. Class Diagram

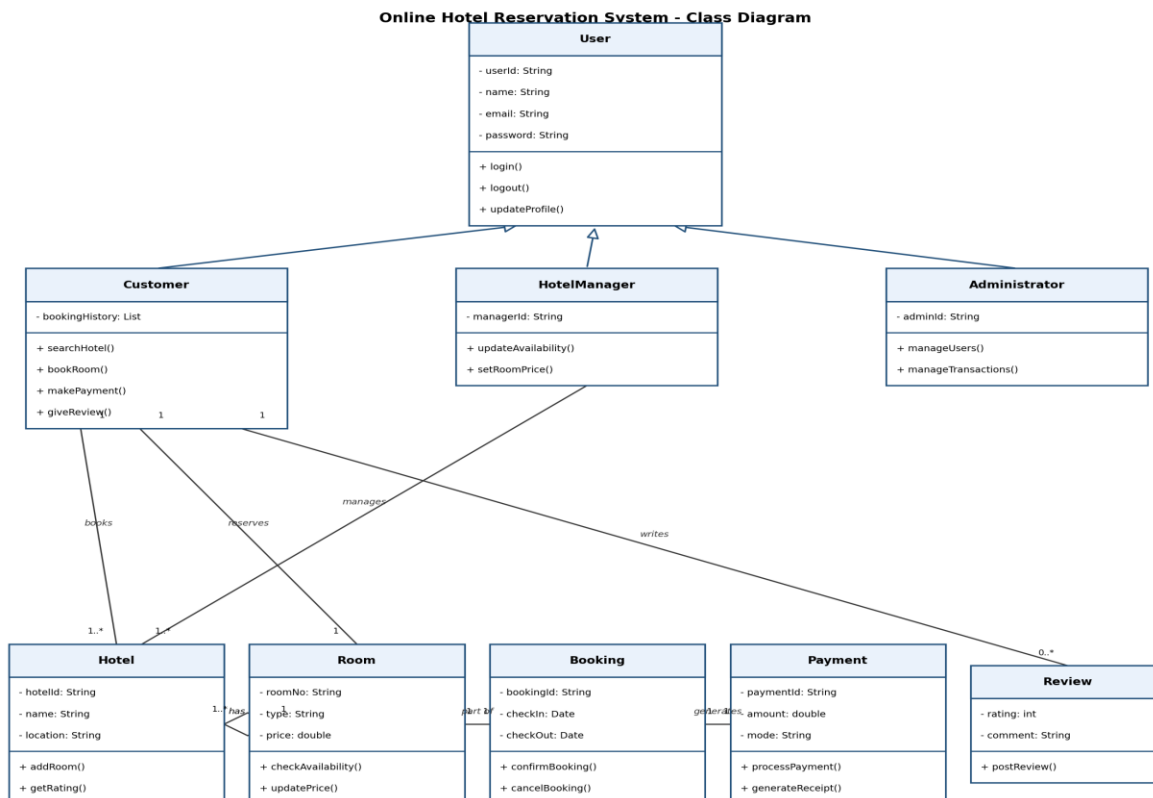


Fig 1.1: Class Diagram - Online Hotel Reservation System

3. Key Relationships

- Generalization — Customer, HotelManager, and Administrator inherit from User.
- Association — Customer books Hotel (1 : 1..*); Customer reserves Room (1 : 1).
- Aggregation — Hotel has Room (1 : 1..*), since Rooms can conceptually be regrouped without the Hotel record disappearing.
- Association — Room is part of a Booking (1 : 1); Booking generates a Payment (1 : 1).
- Association — Customer writes Review (1 : 0..*); HotelManager manages Hotel (1 : 1..*).

4. Application of Object-Oriented Principles

Encapsulation

Each class hides its internal data behind private attributes and exposes controlled access through public methods. For example, Room keeps price and isAvailable as private fields and only allows changes through updatePrice() and checkAvailability(), so a Customer cannot directly overwrite pricing — this protects data integrity across simultaneous bookings.

Inheritance

Customer, HotelManager, and Administrator all inherit common fields (userId, name, email, password) and behaviors (login(), logout(), updateProfile()) from the base class User. This avoids duplicated code and lets each subclass add only the behavior specific to its role — Customer adds bookRoom(), HotelManager adds setRoomPrice(), Administrator adds manageUsers().

Polymorphism

The login() method behaves identically at the User level but can be overridden — a Customer's login redirects to the booking dashboard, a HotelManager's login redirects to the room-management console, and an Administrator's login redirects to the admin panel. The system calls the same method name on a User reference, and the correct behavior executes based on the actual object type at runtime.

5. Recommended SDLC Model

Model: Incremental Model

- The system has clearly separable functional modules — search/booking, payment, room-management, and administration — that can each be built and delivered as a working increment.
- Early increments (search and booking) can go live and gather real customer feedback while later increments (payments, reviews, admin reporting) are still in development.
- Requirements such as payment gateway integration or new review features are likely to evolve after initial rollout, and the incremental model absorbs such changes without reworking the whole system.
- Risk is reduced because critical modules (booking, payment) are tested and stabilized in early increments before less critical features are added.

Question 2: University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system. Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

1. Identified Classes

- Person (base class)
- Student
- Faculty
- ExaminationStaff
- Administrator
- Course
- Enrollment
- Examination
- Result

2. Class Diagram

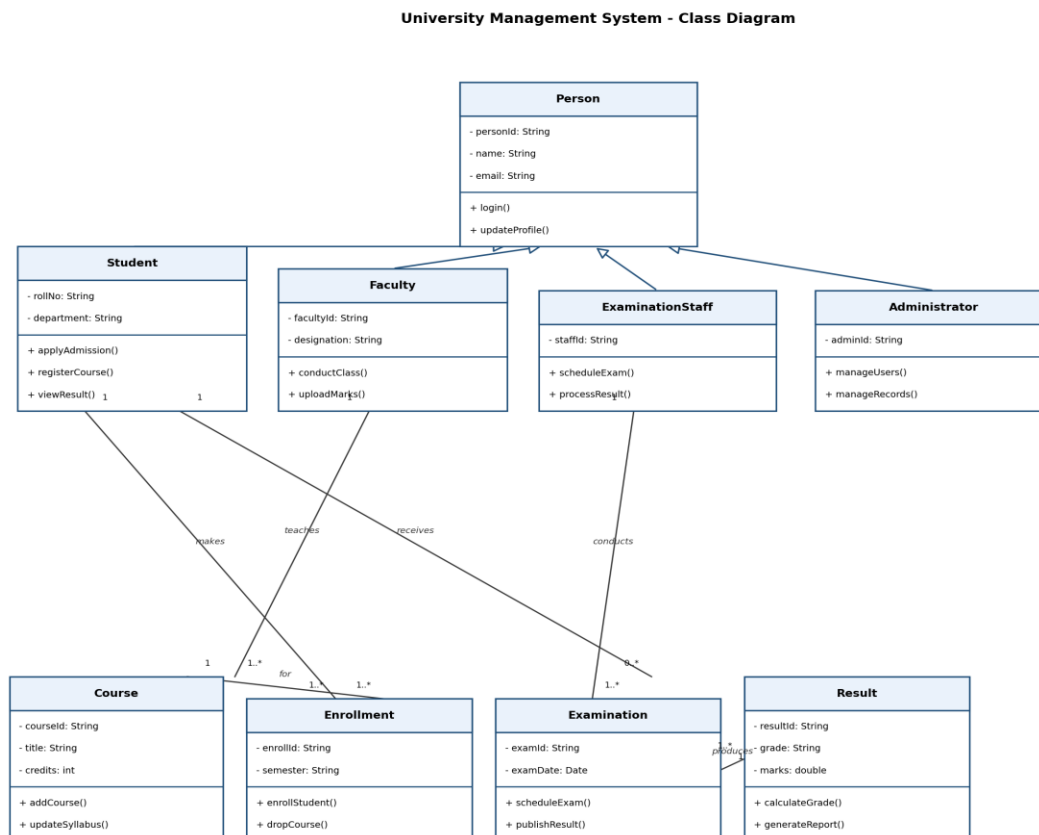


Fig 2.1: Class Diagram - University Management System

3. Key Relationships

- Generalization — Student, Faculty, ExaminationStaff, and Administrator inherit from Person.
- Association — Student makes Enrollment (1 : 1..*); Enrollment is for a Course (1..* : 1).
- Association — Faculty teaches Course (1 : 1..*); ExaminationStaff conducts Examination (1 : 1..*).
- Association — Examination produces Result (1 : 1..*); Student receives Result (1 : 0..*).

4. Application of Object-Oriented Principles

Encapsulation

Sensitive data such as a Student's marks or a Faculty member's designation is kept private inside its class and exposed only through defined methods like `viewResult()` or `uploadMarks()`, preventing unauthorized users from editing academic records directly.

Inheritance

Student, Faculty, ExaminationStaff, and Administrator all inherit common identity attributes (`personId`, `name`, `email`) and behaviors (`login()`, `updateProfile()`) from the base class `Person`, while each adds its own role-specific operations — Student adds `registerCourse()`, Faculty adds `uploadMarks()`, ExaminationStaff adds `scheduleExam()`.

Polymorphism

A generic method such as `generateReport()` behaves differently depending on the caller: for a Student it produces a personal transcript, for Faculty it produces a class performance report, and for an Administrator it produces an institution-wide statistical report — the same call, resolved differently by object type.

5. Recommended SDLC Model

Model: Spiral Model

- A university system integrates several risk-prone, interdependent modules — admissions, registration, examinations, and results — where errors in one module (e.g. result computation) can seriously affect others.
- The Spiral Model's repeated risk-analysis phases allow each module to be prototyped, evaluated, and refined before full-scale implementation, which suits a system that must be highly accurate and auditable.
- Requirements from different stakeholders (students, faculty, exam cell, administration) often surface incrementally through consultation, and the spiral's iterative cycles accommodate this evolving, multi-stakeholder input.
- Because academic systems must be extensively validated (grading rules, eligibility criteria) before go-live, the spiral's built-in evaluation phase at every cycle reduces the risk of costly post-deployment errors.

Question 3: Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory. Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

1. Identified Classes

- User (base class)
- Customer
- Vendor
- Administrator
- Product
- Order
- Payment
- Delivery
- Inventory

2. Class Diagram

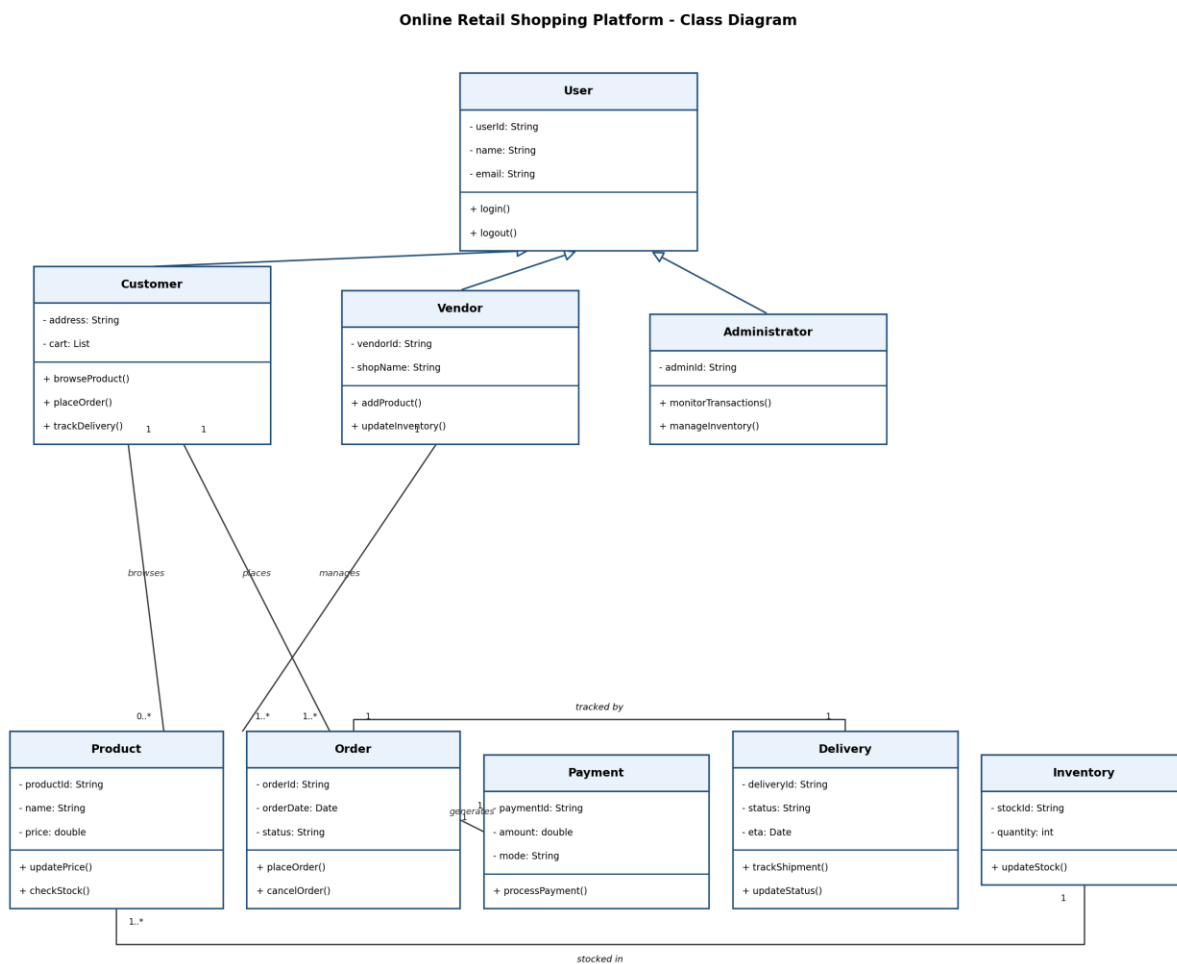


Fig 3.1: Class Diagram - Online Retail Shopping Platform

3. Key Relationships

- Generalization — Customer, Vendor, and Administrator inherit from User.
- Association — Customer browses Product (1 : 0..*); Customer places Order (1 : 1..*).
- Association — Vendor manages Product (1 : 1..*); Order generates Payment (1 : 1).
- Association — Order is tracked by Delivery (1 : 1); Aggregation — Product is stocked in Inventory (1..* : 1).

4. Application of Object-Oriented Principles

Encapsulation

Each entity manages its own state internally — for instance, Order keeps status private and only changes it through placeOrder() or cancelOrder(), and Inventory only exposes quantity through updateStock(), so stock levels cannot be corrupted by direct external edits from unrelated modules.

Inheritance

Customer, Vendor, and Administrator inherit shared identity data and login()/logout() behavior from the base class User, while each extends it with distinct responsibilities: Customer adds placeOrder(), Vendor adds updateInventory(), Administrator adds monitorTransactions().

Polymorphism

A shared method such as processPayment() can behave differently depending on the payment mode object passed to it (card, wallet, UPI, netbanking) — the platform calls the same interface, and the correct payment logic executes based on the actual payment-mode implementation used at runtime.

Modular Software Design through OOP

Modularity is achieved by giving each real-world entity its own class with a single, well-defined responsibility: Product only manages catalog data, Order only manages order state, Payment only manages transaction processing, Delivery only manages shipment tracking, and Inventory only manages stock counts. Because these classes communicate through public methods rather than shared internal data, any one module — for example, swapping the Payment class to support a new payment gateway — can be modified or replaced without touching Order, Delivery, or Inventory. Inheritance further reduces duplication by centralizing common user behavior in the User base class, while polymorphism lets the platform treat different vendors, payment modes, or delivery partners uniformly through a common interface.

5. Recommended SDLC Model

Model: Incremental Model

- The platform naturally splits into independent modules (catalog browsing, ordering, payment, delivery tracking, inventory) that map directly to the object-oriented classes identified above, so each can be delivered as a working increment.
- The Waterfall Model was considered but rejected — it assumes fixed requirements up front, which does not suit an e-commerce platform where new payment gateways, delivery partners, or catalog features are added after launch.
- The Spiral Model was considered but is heavier than needed here — the retail platform does not carry the same high-risk, safety-critical profile (e.g. financial regulation, life-safety data) that justifies spiral's repeated formal risk-analysis overhead.

- The Incremental Model was selected because it lets the core shopping flow (browse → order → pay) go live first and generate revenue while vendor tools, delivery tracking, and admin monitoring are added in later increments, each independently tested against the modular class design.

Question 4: Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application. Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

1. Identified Classes

- User (base class)
- Citizen
- Administrator
- WasteBin
- Sensor
- CollectionVehicle
- IssueReport

2. Class Diagram

Smart Waste Management System - Class Diagram

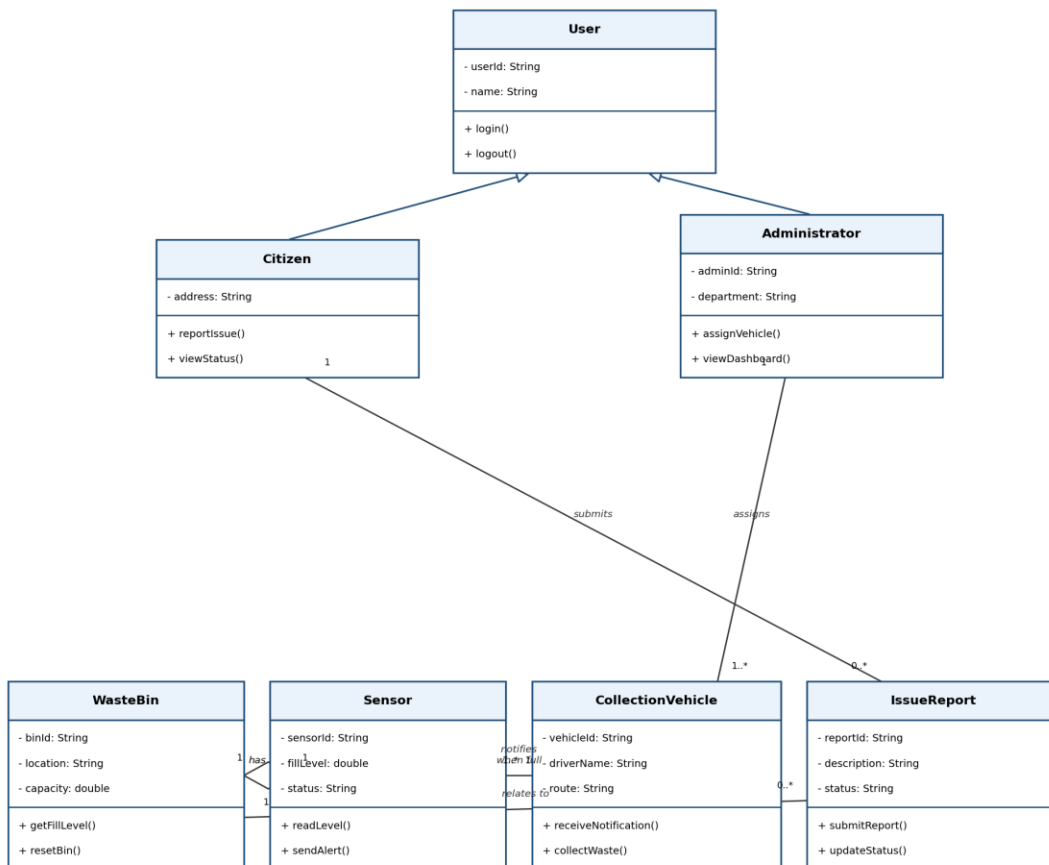


Fig 4.1: Class Diagram - Smart Waste Management System

3. Key Relationships

- Generalization — Citizen and Administrator inherit from User.
- Aggregation — WasteBin has Sensor (1 : 1).
- Association — Sensor notifies CollectionVehicle when full (1 : 1..*); Administrator assigns CollectionVehicle (1 : 1..*).
- Association — Citizen submits IssueReport (1 : 0..*); IssueReport relates to WasteBin (0..* : 1).

4. Application of Object-Oriented Principles

Encapsulation

The Sensor class keeps fillLevel and status private, exposing them only through readLevel() and sendAlert(), so no external module can falsify sensor readings — critical for a system where automated decisions (dispatching a vehicle) depend on trustworthy data.

Inheritance

Citizen and Administrator inherit common attributes (userId, name) and behavior (login(), logout()) from the base class User, while Citizen adds reportIssue()/viewStatus() and Administrator adds assignVehicle()/viewDashboard(), avoiding duplicated authentication logic.

Polymorphism

The method sendAlert() in Sensor can be overridden per sensor/bin type — a standard bin sensor sends a simple threshold alert, while a hazardous-waste bin sensor overrides it to send a high-priority alert with different escalation rules — the collection system calls the same method uniformly regardless of bin type.

5. Recommended SDLC Model

Model: Agile Model

- The system depends on real-time IoT sensor data and hardware integration, both of which are refined through frequent testing cycles with actual hardware — Agile's short sprints allow the sensor-integration module to be tested and adjusted continuously.
- Requirements evolve as the municipality observes real deployment (e.g. adding new bin types, adjusting alert thresholds, new citizen-reporting categories), and Agile's iterative sprints accommodate this without disrupting already-delivered modules.
- Citizen-facing features (the issue-reporting app) benefit from early and frequent user feedback, which Agile supports through incremental releases and review at the end of each sprint.
- Because sensor hardware, notification logic, and vehicle-dispatch logic are relatively independent modules, they can be developed and iterated on in parallel sprints by different sub-teams.